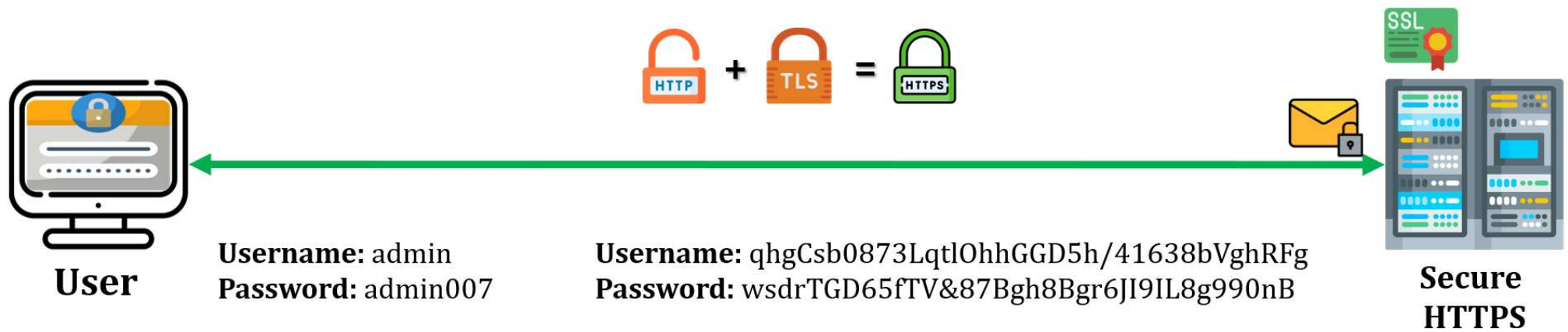
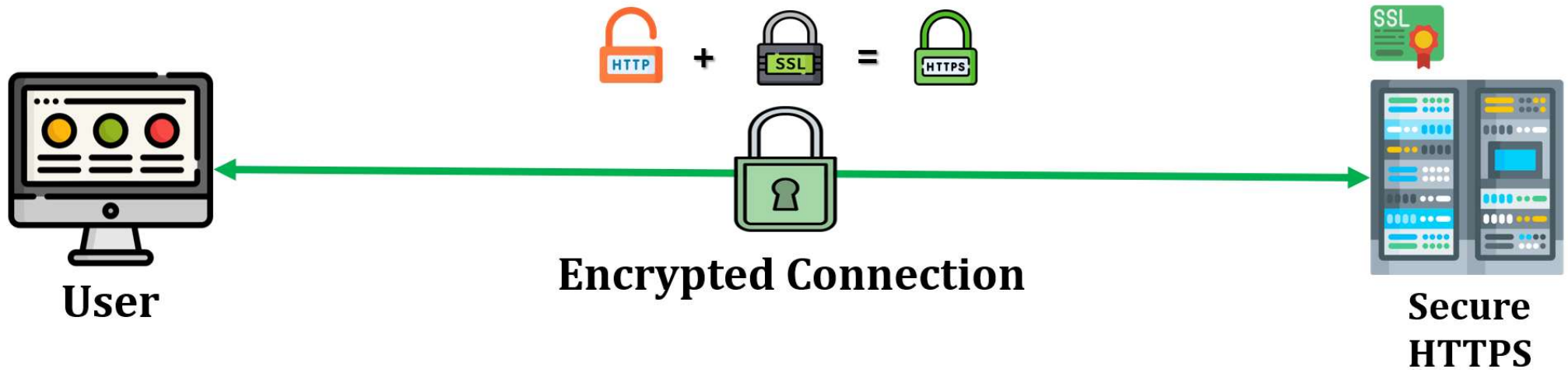


Protocoles SSL/TLS

Protocoles HTTPS



Protocoles HTTPS

Lorsque HTTPS est utilisé, les éléments suivants sont cryptés pendant la communication :

- **URL** du document demandé
- **Contenu du document**
- **Contenu des formulaires du navigateur** (remplis par l'utilisateur du navigateur)
- **Cookies** envoyés du navigateur au serveur et du serveur au navigateur
- **Contenu des en-têtes HTTP**

Protocoles HTTPS

La connexion HTTPS se déroule en trois phases :

- **Initiation de la connexion.**
- **Transfert de donnée.**
- **Fermeture de la connexion.**

Protocoles SSL/TLS

SSL

- **Secure Sockets Layer** (couche de sockets sécurisés).
- Il a été conçu pour fournir **des services de sécurité et de compression aux données générées par la couche application.**
- Les données reçues de l'application sont compressées (facultatif), **signées et cryptées avant de passer à la couche suivante.**

Protocoles SSL/TLS

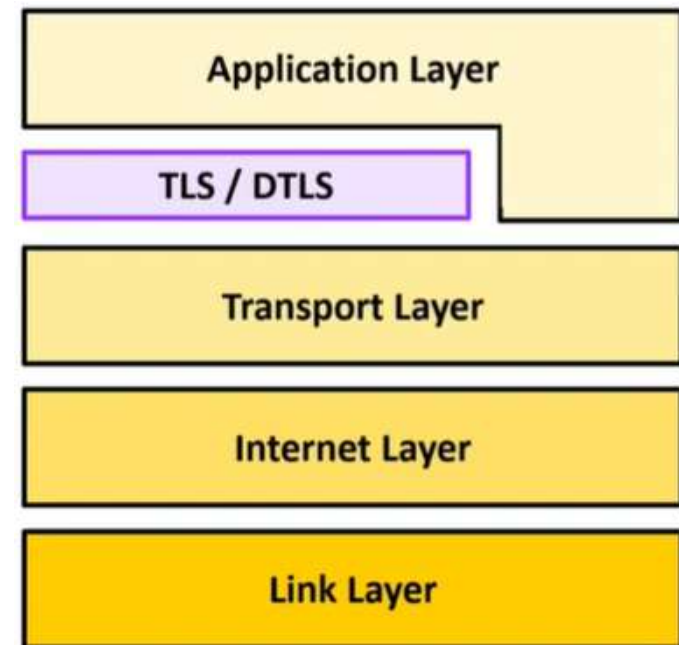
TLS

- **Transport Layer Security** (sécurité de la couche transport).
- TLS permet une **communication** sécurisée entre les navigateurs web et les serveurs. Ainsi, la **connexion** elle-même est sécurisée parce que la cryptographie symétrique est utilisée pour chiffrer les données transmises.
- Les clés sont générées de manière unique pour chaque connexion et sont basées sur un secret partagé négocié au début de la session : « **handshake** » TLS.

Protocoles SSL/TLS

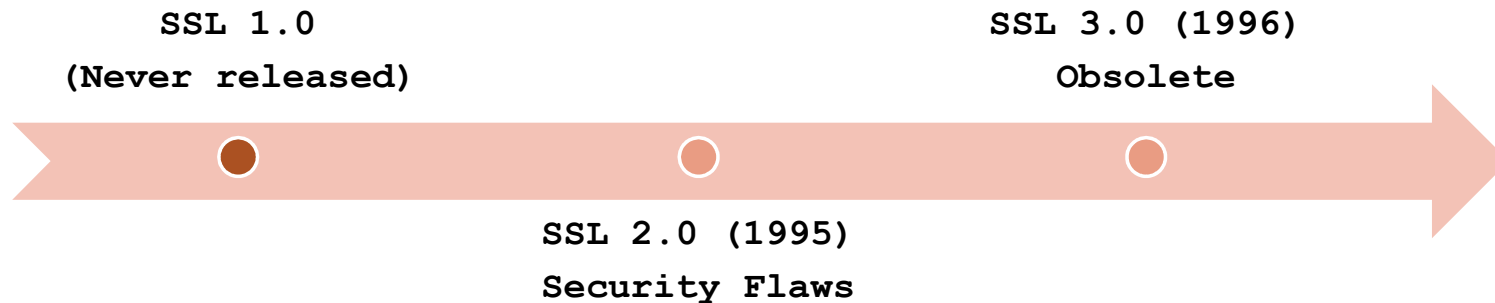
DTLS

- **Datagram Transport Layer Security** DTLS, qui s'exécute sur UDP.
- La dernière version de DTLS est DTLS 1.3, qui est basée sur TLS 1.3, et offre des garanties de sécurité.
- TLS et DTLS sont placés entre la couche d'application et la couche de transport dans le modèle TCP/IP.

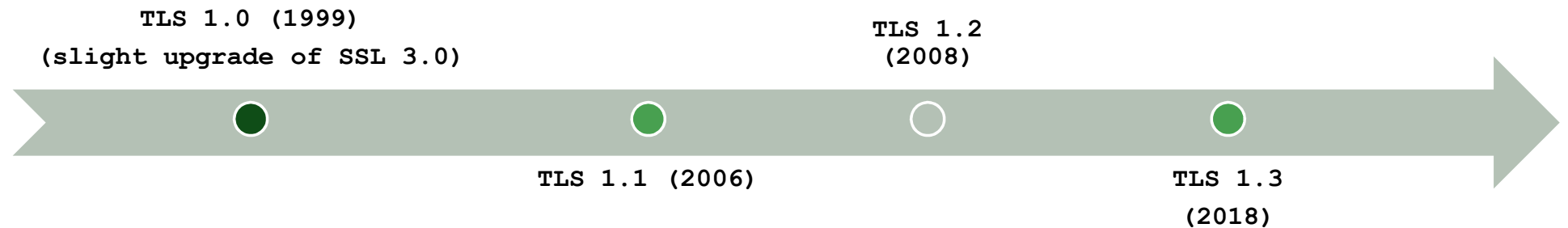


Protocoles SSL/TLS

Secure Socket Layer (SSL) – Développé à l'origine par Netscape



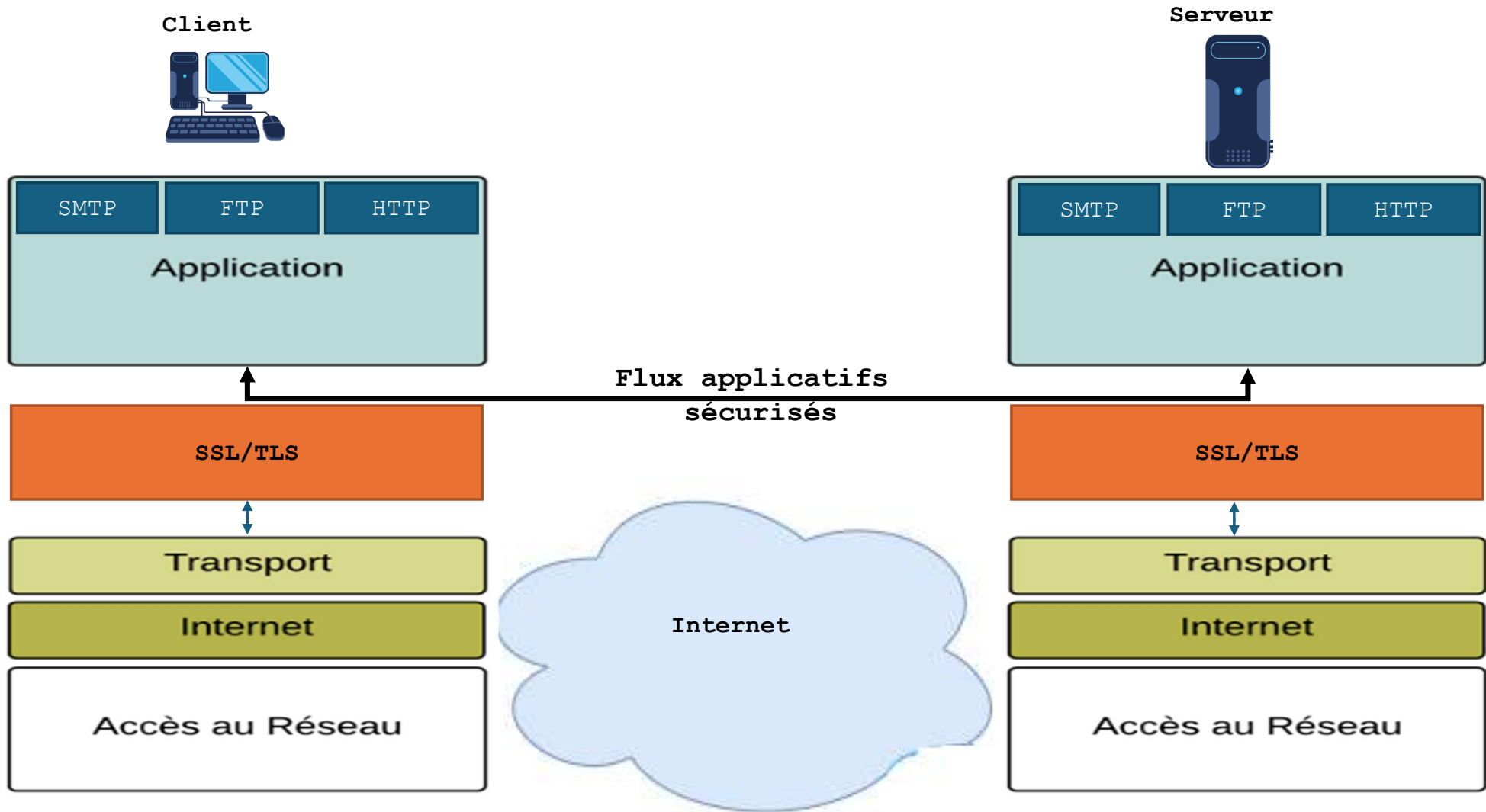
Transport Layer Security(TLS) – Standardisé par l'IETF



Protocoles SSL/TLS

Year	What	Who	Status
1994	SSL 1.0	Netscape	Unpublished
1995	SSL 2.0	Netscape	Deprecated 2011
1996	SSL 3.0	Netscape	Deprecated 2015
1999	TLS 1.0 [2]	IETF	Deprecated 2020
2006	TLS 1.1 [3]	IETF	Deprecated 2020
2008	TLS 1.2 [4]	IETF	Recommended minimum version
2018	TLS 1.3 [5]	IETF	Recommended version

Protocoles SSL/TLS



Protocoles SSL/TLS

Emplacement de SSL/TLS

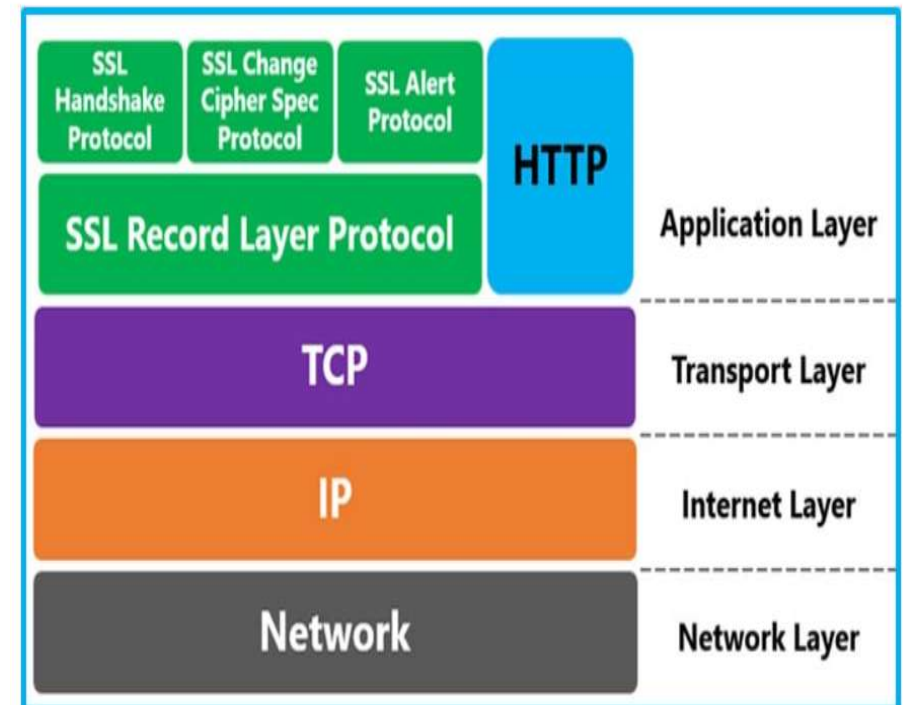
- TLS appartiennent à la **couche de transport** qui fournit une sécurité de bout en bout pour les applications qui utilisent un protocole de couche de transport fiable tel que TCP.
- TLS principalement situés dans la couche transport mais *ils peuvent également être considéré comme fonctionnant entre la couche transport et la couche application.*



Protocoles SSL/TLS

Les protocoles SSL/TLS : se composent de 4 protocoles :

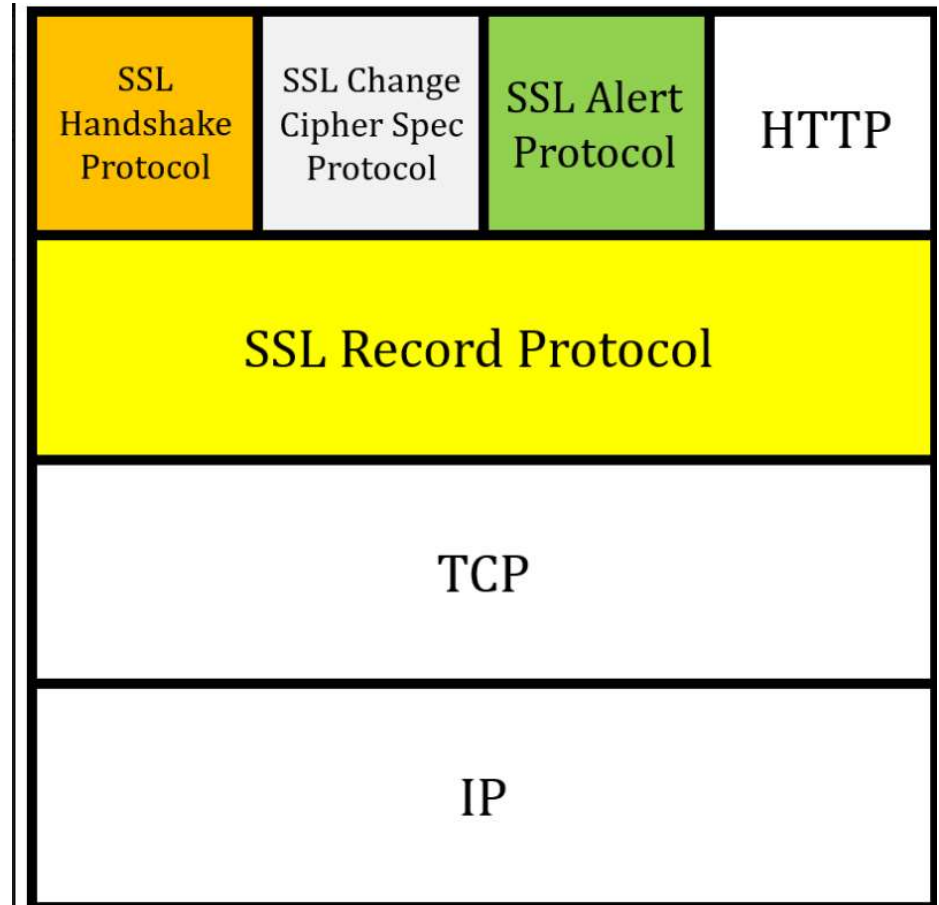
- Protocole Handshake
- Protocole SSL change Cipher
- Protocole SSL Alert
- Protocole Record layer



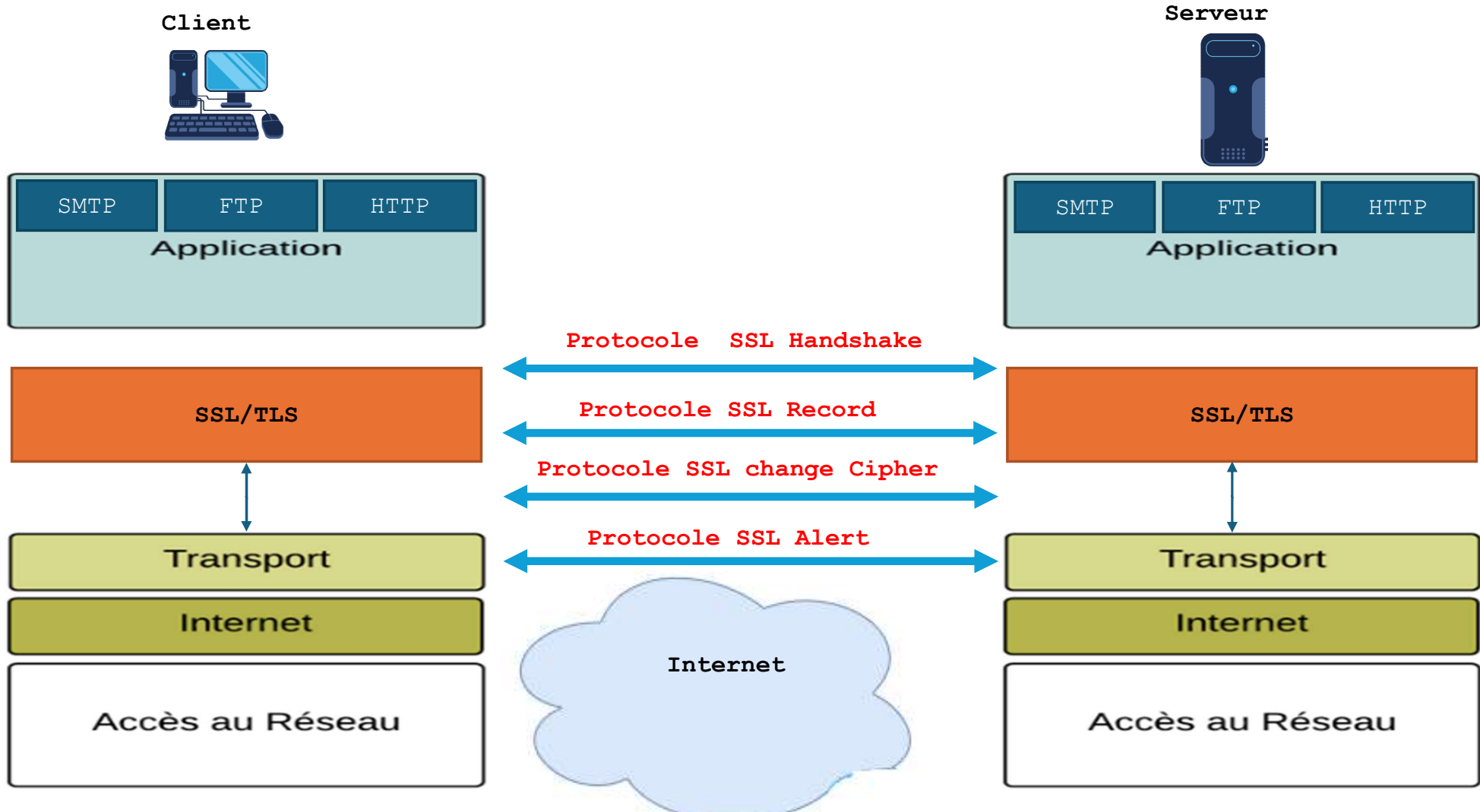
Protocoles SSL/TLS

Deux couches basiques :

- **Le protocole SSL Record :**
fournit des services de sécurité de base.
- **Handshake, changement de spécification de chiffrement, protocole d'Alerte.**

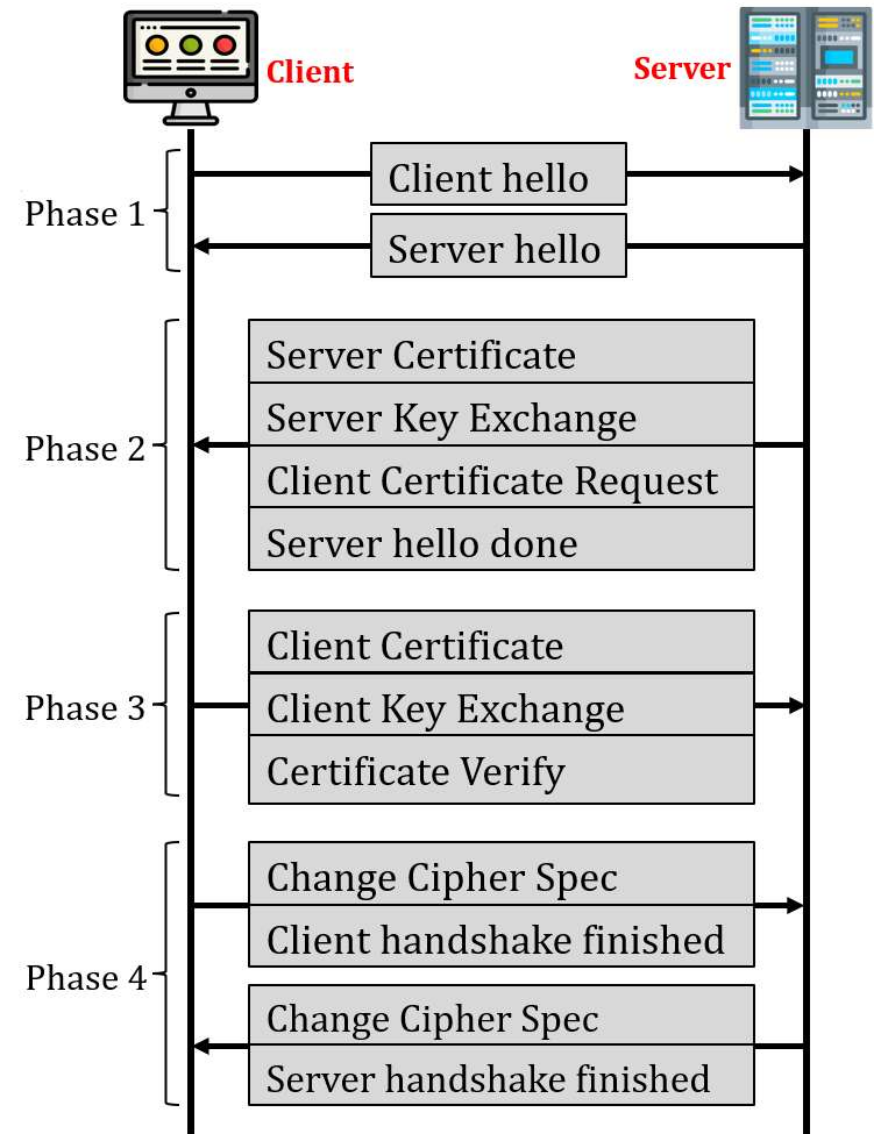


Protocoles SSL/TLS



Protocole handshake

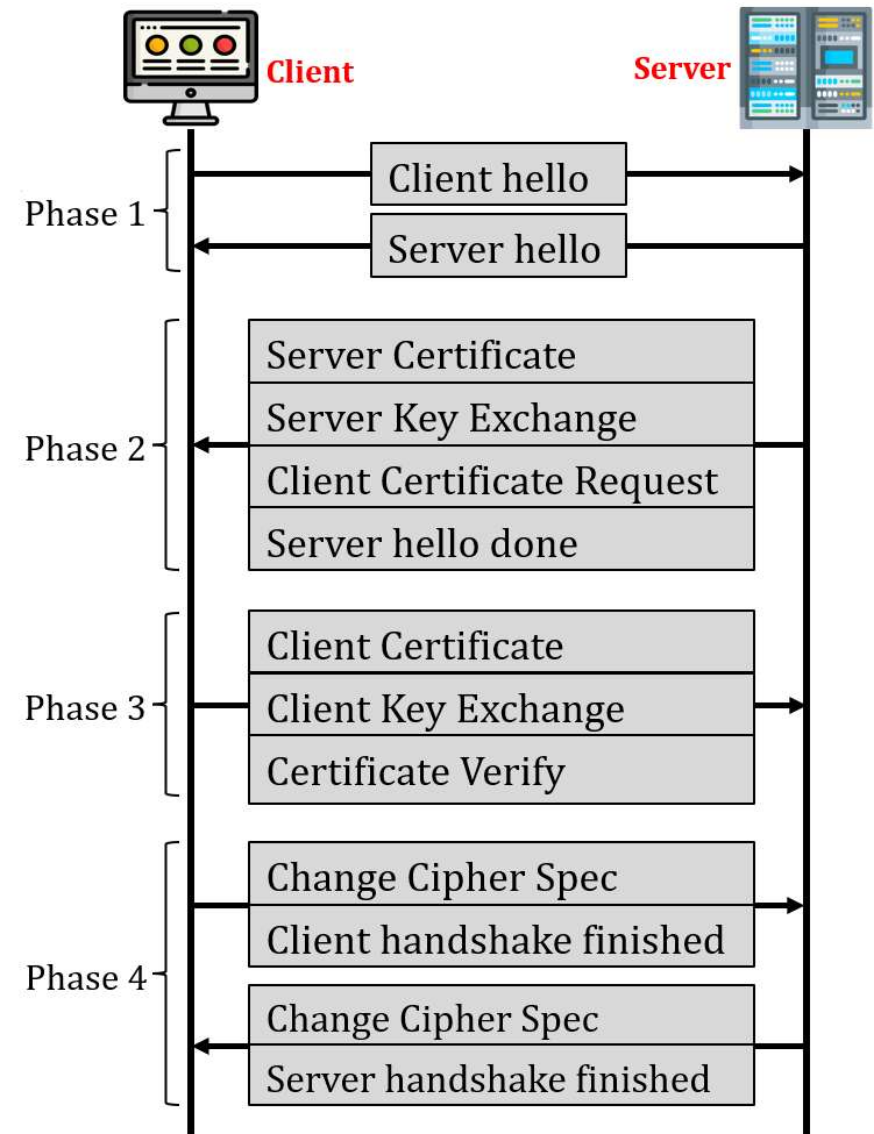
- **Protocole handshake** utilise des **messages** pour :
 - Négocier la suite de chiffrement.
 - Authentifier le serveur par rapport au client
 - Authentifier le client par rapport au serveur.
- Établissement de la connexion.



Protocole handshake

Les 4 phases de la réalisation du Protocole handshake :

- **Phase 1 : Établissement de la capacité de sécurité**
- **Phase 2 : échange de clés et authentication du serveur**
- **Phase 3 : échange de clés et authentication du client**
- **Phase 4 : Finalisation**



Protocole handshake

Phase 1 : Établissement de la capacité de sécurité

- Le client et le serveur annoncent leurs capacités de sécurité et choisissent celles qui leur conviennent.
- Deux messages sont échangés :
 - **ClientHello**
 - **ServerHello**

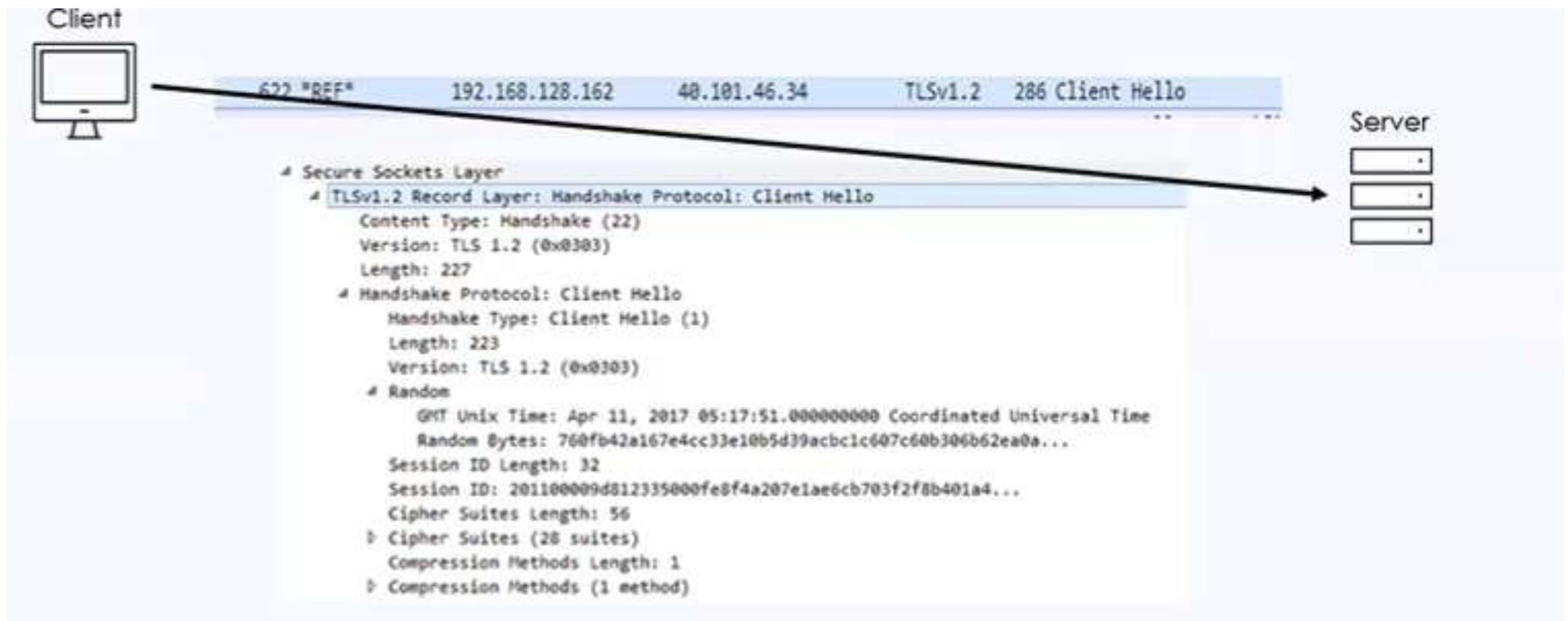
Protocole handshake

ClientHello :

- **Le numéro de version TLS** le plus élevé que le client peut prendre en charge
- **Un nombre aléatoire de 32 octets** qui sera utilisé pour la génération de la clé secrète principale.
- **Un identifiant de session.**
- **Une suite de chiffrement** qui définit la liste des algorithmes que le client peut prendre en charge.
 - **Premier choix : TLS_RSA_WITH_AES_128_GCM_SHA256**
 - **Deuxième choix : TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384**
 - ...
- **Une liste de méthodes de compression** que le client peut prendre en charge (obsolète).

Protocole handshake

ClientHello :



Protocole handshake

ServerHello

- Un numéro de version TLS.
- Un nombre aléatoire de 32 octets qui sera utilisé pour la génération de la clé secrète principale.
- Un identifiant de session.
- Le jeu de chiffrement sélectionné dans la liste des clients.
 - Par exemple : **Premier choix : TLS_RSA_WITH_AES_128_GCM_SHA256**
- La méthode de compression sélectionnée dans la liste des clients (obsolète).

TLS_RSA_WITH_AES_128_GCM_SHA256

TLS : Protocole utilisé.

RSA : Algorithme pour l'échange de clés et l'authentification.

AES_128_GCM : Algorithme de chiffrement symétrique (AES en mode GCM, clé de 128 bits).

SHA256 : Algorithme de hachage pour garantir l'intégrité.

Protocole handshake

ServerHello



```
4 Handshake Protocol: Server Hello
  Handshake Type: Server Hello (2)
  Length: 85
  Version: TLS 1.2 (0x0303)
  Random
  Session ID Length: 32
  Session ID: c228000026a2fe6ea500d795dbd0f368993581e978a8b3ba...
  Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA256 (0xc027)
  Compression Method: null (0)
```

Protocole handshake

Phase 2 : Échange de clés et authentification du serveur

- **Certificat** : Si nécessaire, le serveur envoie un message de certificat pour s'authentifier.
- **ServerKeyExchange** : Il inclut sa contribution au secret pre-master (pré-maître)
- **CertificateRequest (demande de certificat)** : Le serveur peut demander au client de s'authentifier, il envoie donc ce message dans la phase 2 pour obtenir une certification du client dans la phase 3
- **ServerHelloDone** : Ce dernier message signale au client que la phase 2 est terminée et qu'il doit entamer la phase 3.

Protocole handshake

Phase 3 : Échange de clés et authentification du client

- **Certificat** : Pour se certifier auprès du serveur, le client envoie un message de certificat.
- **ClientKeyExchange** : Il inclut sa contribution au secret pré-maître.
- **Vérification du certificat** : Le client doit prouver qu'il possède la clé privée liée à son certificat pour éviter les usurpations.

Protocole handshake

Phase 4 : Finalisation

Client

- **ChangeCipherSpec** : Le client envoie un message **ChangeCipherSpec** pour indiquer qu'il a déplacé l'ensemble de la suite de chiffrement et les paramètres de l'état en attente à l'état actif.
- **Finished** : Il est envoyé par le client. Il s'agit d'un message Finished qui annonce la fin du protocole de handshaking par le client.

• Serveur

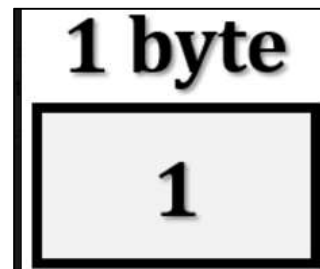
- **ChangeCipherSpec** : Le serveur envoie un message ChangeCipherSpec pour montrer qu'il a déplacé l'ensemble de la suite de chiffrement et les paramètres de l'état en attente à l'état actif.
- **Finished** : ce message est envoyé par le serveur. Il s'agit d'un message Finished qui annonce que la fin du protocole de prise de contact est totalement terminée.

Protocole SSL change Cipher (CCS)

Le protocole de changement de spécification de chiffrement (CCS)

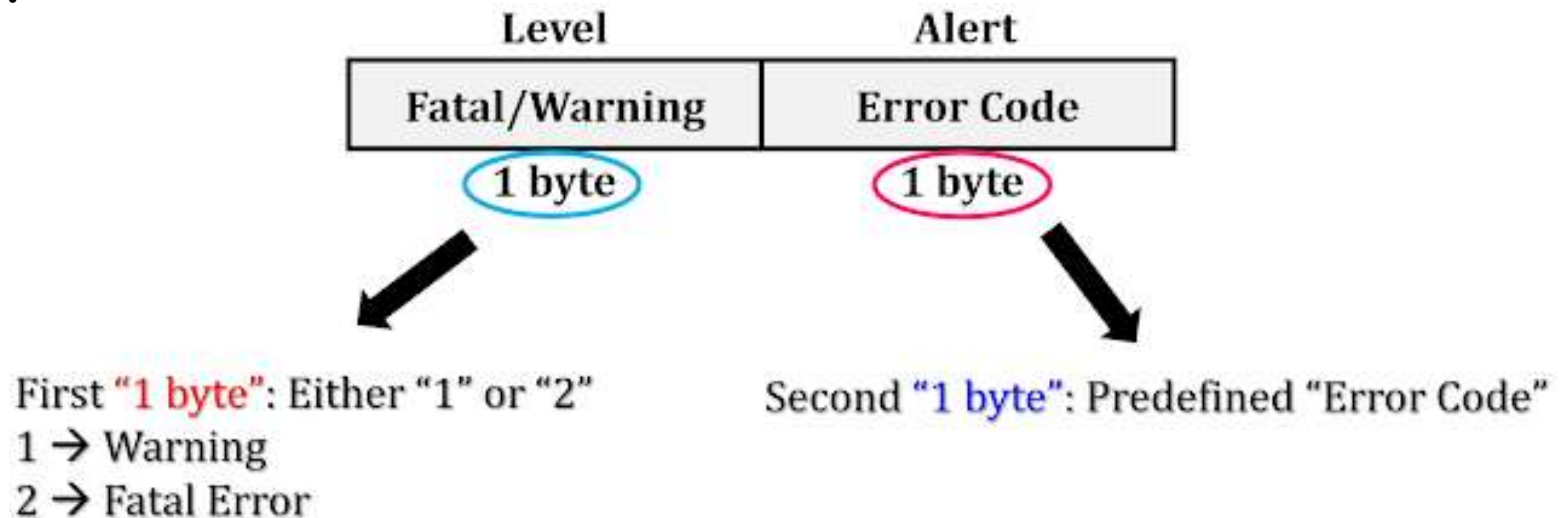
- Utilisé pour modifier le chiffrement utilisé par le client et le serveur.
- Il est normalement utilisé dans le cadre du processus de handshake pour passer à un chiffrement à clé symétrique.
- Le protocole CCS est un message unique qui indique à un pair que l'expéditeur souhaite passer à un nouveau jeu de clés, qui sont ensuite créées à partir des informations échangées dans le cadre du protocole de handshake
- Ce protocole consiste en un message unique composé d'un seul octet.

Un Octet



Protocole SSL Alert

- Les messages d'alerte indiquent la gravité du message et une description de l'alerte.
- L'utilisation principale de ce protocole est de signaler la cause de l'échec.
- Les changements d'état comprennent des éléments tels qu'une condition d'erreur comme un message invalide reçu ou un message qui ne peut pas être décrypté, ainsi que des éléments comme la fermeture de la connexion.



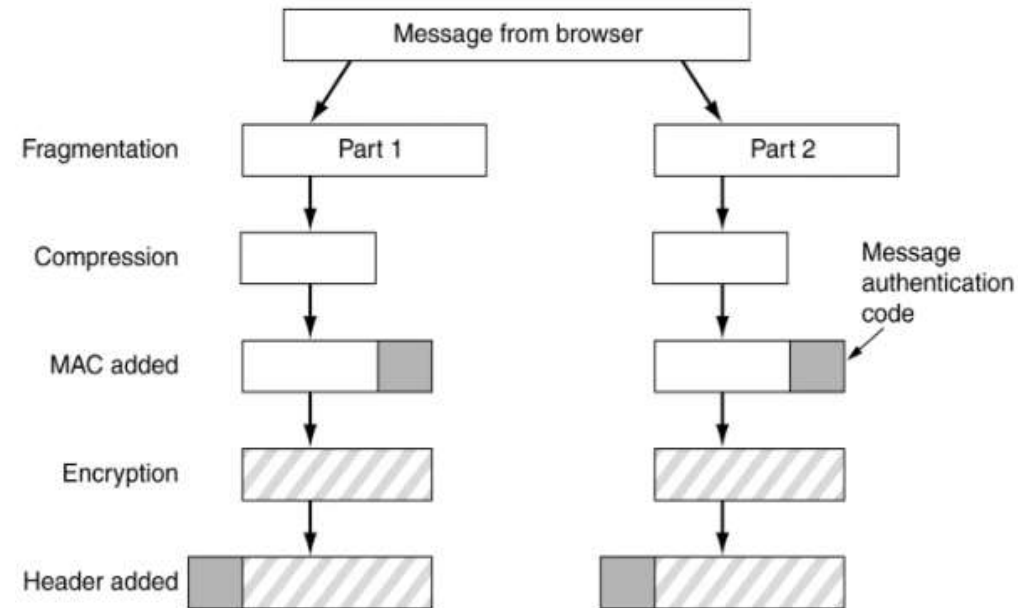
Protocole SSL Alert

Le tableau présente les types de messages d'alerte.

Alert Code	Alert Message	Description
0	close_notify	No more message from sender.
10	unexpected_message	An incorrect message received.
20	bad_record_mac	A wrong MAC received.
30	decompression_failure	Unable to decompress.
40	handshake_failure	Unable to finalize handshake by the sender.
42	bad_certificate	Received a corrupted certificate.
42	No_certificate	Client has no certificate to send to server.
42	certificate_expired	Certificate has expired.

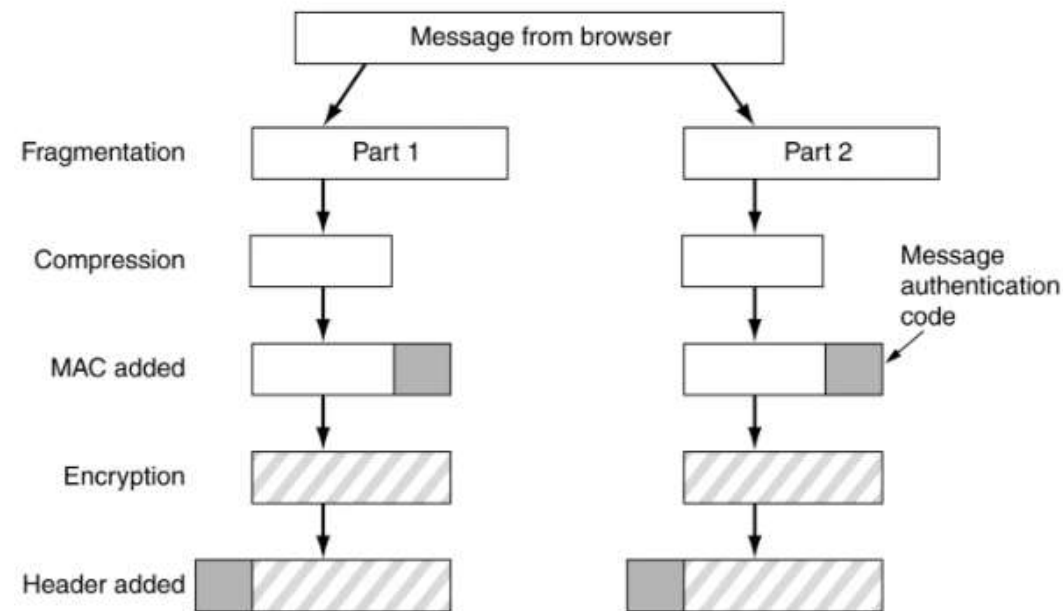
Protocole Record layer

- **Fragmentation** : Le message original qui doit être envoyé est divisé en blocs. La taille de chaque bloc est inférieure ou égale à 214 octets.
- **Compression** : Les blocs fragmentés sont compressés, ce qui est facultatif. Il convient de noter que le processus de compression ne doit pas entraîner la perte des données originales.
- **Ajout de MAC** : Un court élément d'information utilisé pour authentifier un message afin d'en garantir l'intégrité et l'assurance.



Protocole Record layer

- **Cryptage** : L'ensemble des étapes, y compris le message, est crypté à l'aide d'une clé symétrique, mais le cryptage ne doit pas augmenter la taille globale du bloc.
- **Ajout d'un en-tête** : Après toutes les opérations ci-dessus, un en-tête est ajouté au bloc crypté.



Protocole Record layer

Exemple de flux complet

Message original : Le navigateur envoie une requête HTTP.

Fragmentation : La requête est divisée en plusieurs petits blocs (Part 1, Part 2, etc.).

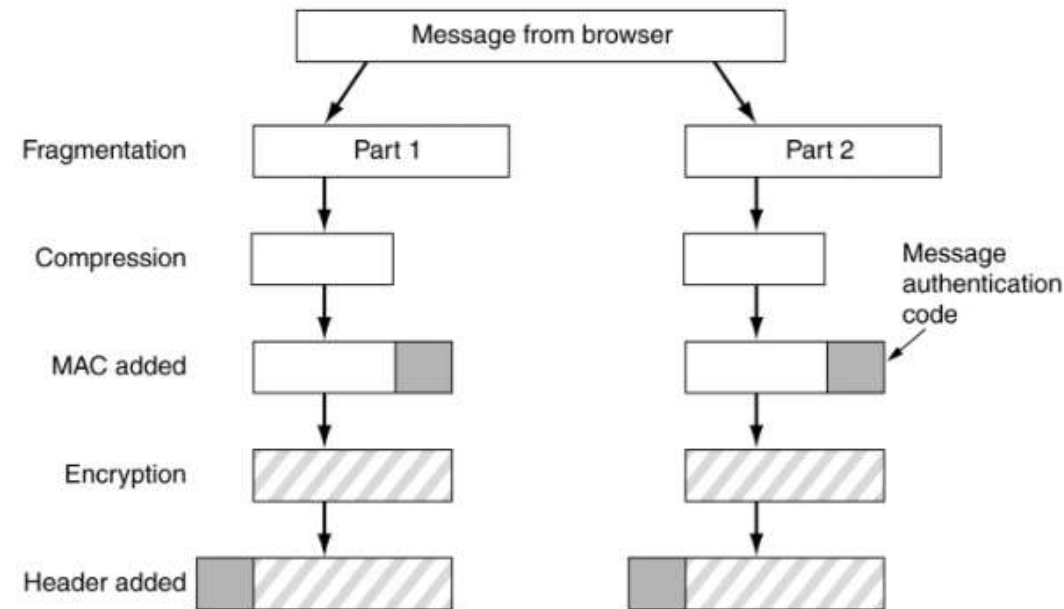
Compression : Chaque bloc est compressé (si la compression est activée).

Ajout du MAC : Un code d'authentification est ajouté à chaque bloc.

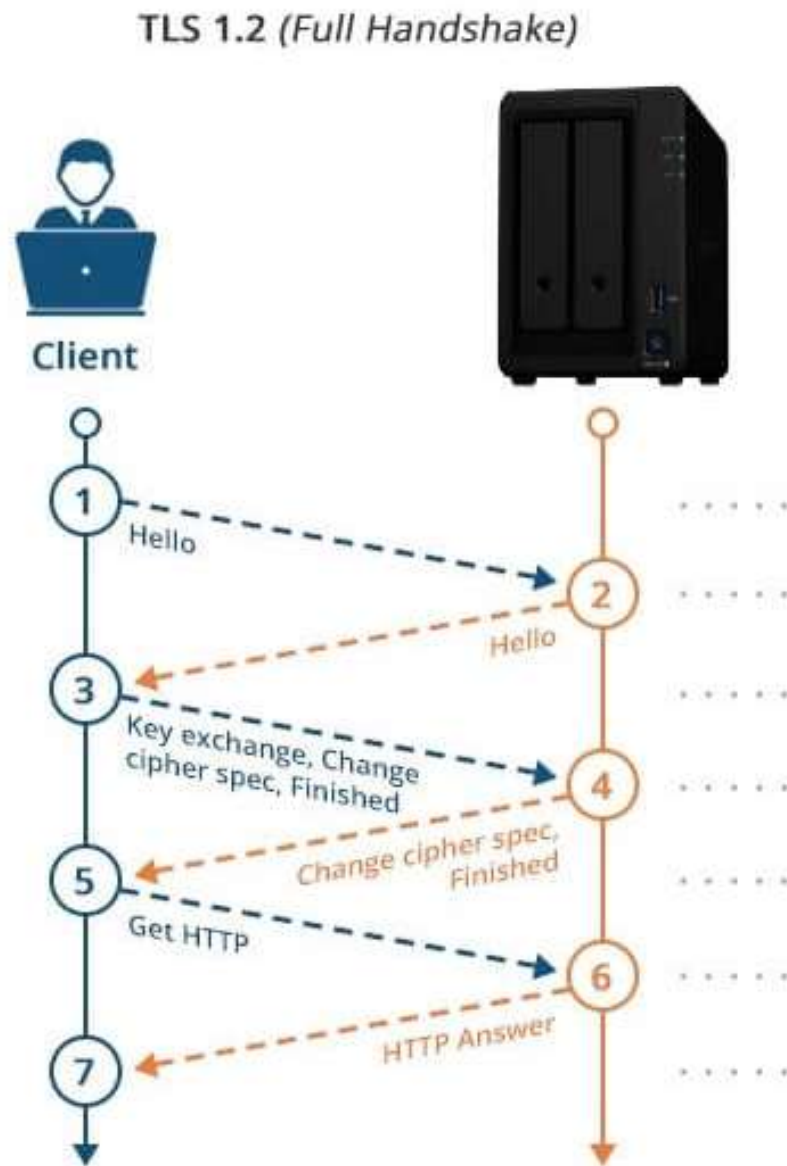
Chiffrement : Les blocs sont chiffrés pour protéger les données.

Ajout de l'en-tête : Un en-tête est ajouté pour indiquer des informations sur le

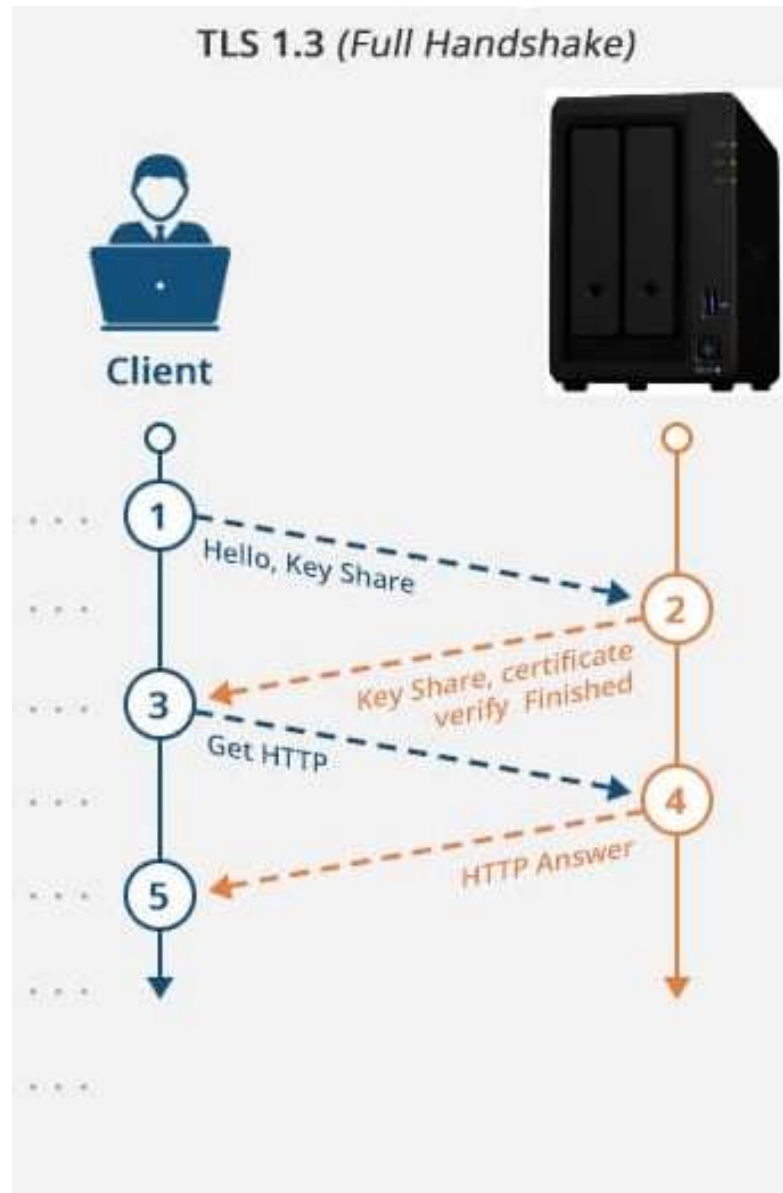
message.



TLS v1.2

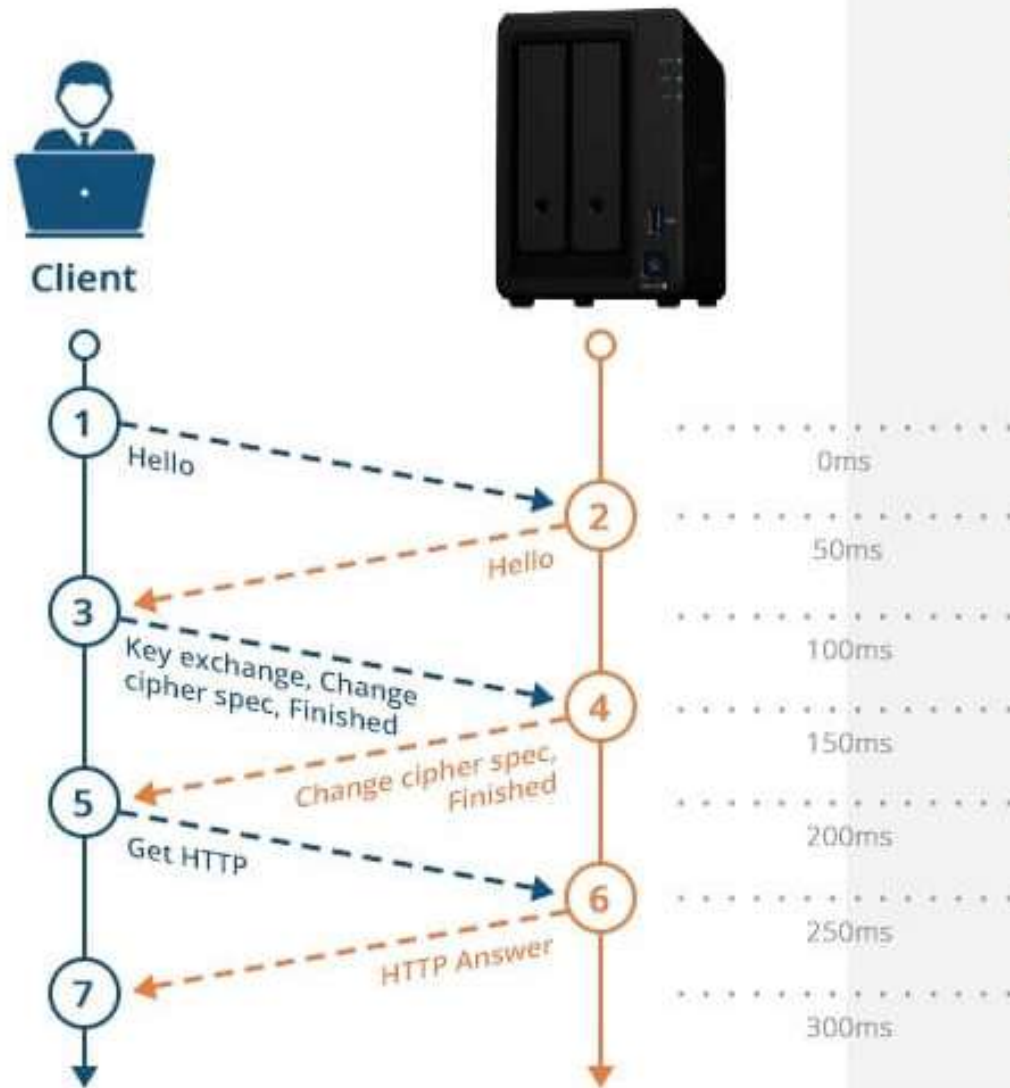


TLS v1.3

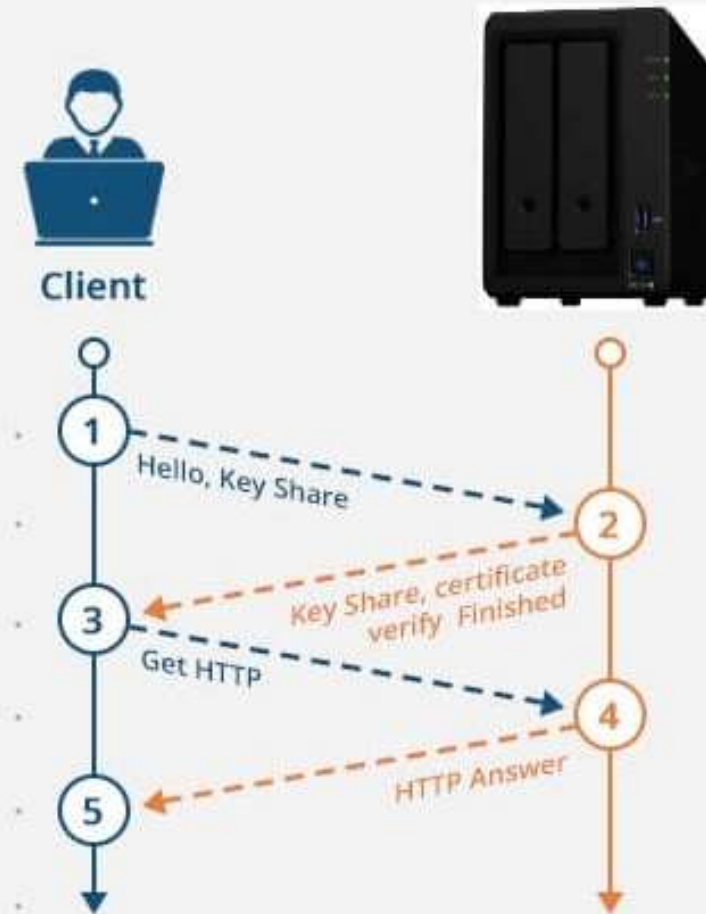


TLS
v1.2
vs
TLS
v1.3

TLS 1.2 (Full Handshake)



TLS 1.3 (Full Handshake)



TLS v1.2 vs TLS v1.3

- **Le RTT (Round-Trip Time)** est une mesure du temps nécessaire pour qu'un signal ou un paquet de données voyage d'un point à un autre (par exemple, d'un client à un serveur) et revienne à l'expéditeur.
- **Dans le contexte de TLS (Transport Layer Security) :**
 - Le RTT est utilisé pour mesurer le nombre d'allers-retours nécessaires pour compléter une poignée de main TLS (l'établissement d'une connexion sécurisée).
 - Chaque aller-retour (RTT) correspond à un échange de messages entre le client et le serveur.

TLS v1.2 vs TLS v1.3

TLS 1.2 :

Nombre de RTT : 2 (par défaut)

Le processus du Handshake dans TLS 1.2 nécessite généralement **2 allers-retours** entre le client et le serveur. Cela inclut :

- **Premier RTT** : Le client envoie un "ClientHello" et le serveur répond avec un "ServerHello", suivis des certificats et des paramètres de clé.
- **Deuxième RTT** : Le client envoie la clé pré-maître chiffrée et termine la négociation, puis le serveur confirme et active le chiffrement.

Cas particulier : Si la **reprise de session** (session resumption) est utilisée (via un **session ID** ou **session ticket**), le nombre de RTT peut être réduit à **1**.

TLS v1.2 vs TLS v1.3

TLS 1.3 :

Nombre de RTT : 1 (par défaut)

TLS 1.3 a été optimisé pour réduire la latence en permettant une poignée de main en **1 seul aller-retour**.

Cela inclut :

- **Premier RTT** : Le client envoie un "ClientHello" avec des propositions de clés (y compris une clé partagée via **ECDHE**), et le serveur répond immédiatement avec un "ServerHello", les certificats, et termine la négociation en activant le chiffrement.
- **Aucun deuxième RTT** : Le chiffrement est activé immédiatement après le premier échange, ce qui réduit la latence.

Cas particulier : Si le client et le serveur ont déjà communiqué auparavant et utilisent une **reprise de session** (via un **PSK - Pre-Shared Key**), la poignée de main peut être effectuée en **0 RTT**, ce qui est idéal pour les applications sensibles à la latence.

TLS v1.2 vs TLS v1.3

Exemple

TLS 1.2

TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA256

TLS 1.3

TLS_AES_256_GCM_SHA384

TLS v1.2 vs TLS v1.3

Exemple

TLS 1.2

- **Suite de chiffrement** : TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA256
- **Échange de clés** : ECDHE (Elliptic Curve Diffie-Hellman Ephemeral)
- **Authentification** : RSA
- **Chiffrement** : AES-256 en mode CBC (Cipher Block Chaining)
- **Algorithme de hachage** : SHA256

TLS 1.3

- **Suite de chiffrement** : TLS_AES_256_GCM_SHA384
- **Chiffrement** : AES-256 en mode GCM (Galois/Counter Mode)
- **Algorithme de hachage** : SHA384

TLS v1.2 vs TLS v1.3

Suppression des algorithmes obsolètes

- TLS 1.3 retire des algorithmes considérés comme faibles ou compromis, notamment :
 - **RSA key exchange** (remplacé par Diffie-Hellman Ephemeral)
 - **MD5 et SHA-1** (vulnérables aux collisions)
 - **Cipher suites basées sur CBC** (vulnérables à des attaques comme BEAST)